

CURSO DE ENGENHARIA DE SOFTWARE / CIÊNCIA DA COMPUTAÇÃO
TEORIA DA COMPUTAÇÃO E LINGUAGENS FORMAIS

**DESENVOLVIMENTO DE SISTEMA UNIFICADO DE
VALIDAÇÃO FORMAL**

Análise e Implementação Empírica dos Três Níveis Estruturais da Hierarquia de Chomsky

Kauã Abrahão de Almeida
Lucas Tiepo de Oliveira
Vitor Gustavo J. De Carvalho

Relatório técnico descritivo apresentado como requisito parcial de avaliação na disciplina de Teoria da Computação. Desenvolvido para analisar na prática as diferenças de escopo computacional, limitações de armazenamento e contagem de passos de execução entre Autômatos Finitos, Autômatos a Pilha e Máquinas de Turing aplicados a cenários reais de engenharia.

Brasil
2026

1. Introdução e Contexto Aplicado

No desenvolvimento de software corporativo, a validação de strings de entrada é crítica para a segurança e estabilidade. Sempre que um sistema avalia se uma entrada satisfaz a regras pré-definidas, executa o papel de um reconhecedor de linguagem formal. Validadores diferem drasticamente na complexidade computacional dos seus algoritmos. Um erro clássico de arquitetura é empregar expressões regulares simples para analisar estruturas que contêm aninhamento dinâmico ou dependências de contexto de longo alcance.

Este relatório apresenta a fundamentação teórica, o projeto mecânico de transição e a análise empírica de um sistema orquestrador unificado contendo três reconhecedores manuais. Cada um representa um nível da Hierarquia de Chomsky: Linguagem Regular (LR), Linguagem Livre de Contexto (LLC) e Linguagem Recursiva (R). O tema aborda a validação de formatos de CPFs, o balanceamento sintático de expressões algébricas e a integridade de dados via espelhamento binário.

2. Linguagem Regular Escolhida: Validação de Formato de Identificador (CPF)

2.1 Descrição e Definição Formal

A linguagem regular compreende o padrão estrito de formatação tipográfica do CPF adotado no Brasil, delimitando-se exclusivamente à validação sintática das posições dos caracteres separadores e blocos numéricos. Uma cadeia pertence a esta linguagem se consistir exatamente em três dígitos numéricos, um ponto, três dígitos, um ponto, três dígitos, um hífen e dois dígitos numéricos terminais. A validação não executa cálculos aritméticos ponderados de dígitos verificadores.

Seja o alfabeto composto por algarismos e pontuações: $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., -\}$. Definimos o conjunto de dígitos como $D = \{0..9\}$. A linguagem formal regular $L_{\{reg\}}$ é expressa por:

$$L_{\{reg\}} = \{ d_1 d_2 d_3 . d_4 d_5 d_6 . d_7 d_8 d_9 - d_{10} d_{11} \mid d_i \in D \text{ ext} \{ \text{para todo } i \mid 1 \leq i \leq 11 \} \}$$

O alfabeto operacional possui 12 símbolos terminais. Cadeias com caracteres alfabéticos, espaços ou pontuações extras são rejeitadas de imediato por violação de alfabeto. Exemplos de cadeias aceitas: **123.456.789-00** e **000.000.000-00**. Exemplos de rejeitadas: **12.34.56-78** (blocos incorretos) e **123.456.78900** (sem hífen).

3. Linguagem Livre de Contexto Escolhida: Expressões Algebricamente Balanceadas

3.1 Descrição e Definição Formal

Esta linguagem descreve a sintaxe de ordenação e balanceamento estrito de parênteses em expressões algébricas. O objetivo é certificar que, para cada abertura (, exista um fechamento correspondente) sequencial posterior. A linguagem permite a intercalação de variáveis minúsculas e operadores matemáticos, contanto que as regras de aninhamento jamais sejam violadas. Um fechamento nunca pode anteceder a abertura correspondente e o sistema não pode encerrar a leitura com escopos abertos.

A linguagem livre de contexto $L_{\{llc\}}$ é gerada a partir da Gramática Livre de Contexto $G = (V, \Sigma_{\{llc\}}, P, S)$, onde $V = \{S, E, T\}$ e o alfabeto de terminais é $\Sigma_{\{llc\}} = \{ (,), +, -, *, /, a..z, 0..9 \}$. As regras de produção P são definidas por:

$$S \rightarrow (S) \mid SS \mid E \mid \epsilon \quad E \rightarrow E + T \mid E - T \mid E * T \mid E / T \mid T$$

$$T \rightarrow a \mid b \mid \dots \mid z \mid 0 \mid \dots \mid 9$$

Para o modelo computacional do Autômato a Pilha (PDA), define-se o alfabeto da fita/pilha $\Gamma = \{X, Z\}$, onde Z é o marcador estático de fundo e X rastreia os escopos ativos. Exemplos válidos: $((x+y)^*z)$ e $((((a))))$. Inválidos: $((a+b))$ e $(.$.

4. Linguagem Recursiva Escolhida: Integridade e Espelhamento de Dados ($w\#w$)

4.1 Descrição e Definição Formal

A linguagem recursiva selecionada modela o problema clássico de duplicação e verificação de integridade de cadeias, representado pelo formato de espelhamento exato $w\#w$. Uma string pertence a esta linguagem se consistir em uma primeira subcadeia w composta por uma sequência de caracteres binários (zeros e uns), seguida obrigatoriamente por um delimitador central especial ($\#$), e concluindo com uma segunda subcadeia que deve ser uma cópia exata, caractere por caractere e na mesma ordem original, da subcadeia w inicial.

$$L_{\{rec\}} = \{ w\#w \mid w \in \{0, 1\}^* \}$$

Onde * representa o Fecho de Kleene sobre o alfabeto binário. Se $w = \epsilon$ (vazia), a string reduz-se exclusivamente ao delimitador central isolado $\#$, que constitui uma entrada válida. O alfabeto de entrada é $\Sigma_{\{rec\}} = \{0, 1, \#\}$ e o alfabeto da fita expande-se para $\Gamma_{\{rec\}} = \{0, 1, \#, X, Y, _\}$, onde X e Y marcam os caracteres já processados. Cadeias aceitas: $101\#101$ e $\#$. Rejeitadas: $101\#100$ e 101101 .

5. Modelos Formais e Transições Estruturadas

5.1 Nível Regular: Tabela Completa de Transições do DFA

O Autômato Finito Determinístico (DFA) opera com um conjunto finito de 15 estados formais, $Q = \{q_0, \dots, q_{14}\}$, onde q_0 é o inicial e o único de aceitação é $F = \{q_{14}\}$.

Estado Origem	Símbolo Lido	Estado Destino	Significado Semântico do Passo
q_0, q_1, q_2	[0-9]	q_1, q_2, q_3	Consome 1º, 2º e 3º dígitos do primeiro bloco do CPF
q_3	.	q_4	Valida obrigatoriamente o primeiro ponto separador
q_4, q_5, q_6	[0-9]	q_5, q_6, q_7	Consome 1º, 2º e 3º dígitos do segundo bloco do CPF
q_7	.	q_8	Valida obrigatoriamente o segundo ponto separador
q_8, q_9, q_{10}	[0-9]	q_9, q_{10}, q_{11}	Consome 1º, 2º e 3º dígitos do terceiro bloco do CPF
q_{11}	-	q_{12}	Valida obrigatoriamente o hífen separador dos verificadores
q_{12}, q_{13}	[0-9]	q_{13}, q_{14}	Consome o 1º e 2º dígitos verificadores finais (Aceitação)

5.2 Nível Livre de Contexto: Transições do PDA

O Autômato a Pilha opera sobre um único estado persistente $Q = \{q_1\}$, delegando a validação ao comportamento LIFO da pilha auxiliar. As transições são as seguintes:

- **Empilhamento (Abertura):** $\delta(q_1, ext\{ '(', Topo \}) \rightarrow (q_1, ext\{ 'X', Topo \})$. Adiciona X à pilha.
- **Desempilhamento (Fechamento):** $\delta(q_1, ext\{ ')', 'X' \}) \rightarrow (q_1, \epsilon)$. Consome o topo X . Se for Z , falha.
- **Transição Neutra:** $\delta(q_1, ext\{ char \}, ext\{ Topo \}) \rightarrow (q_1, ext\{ Topo \})$ para alfanuméricos e operadores neutros.

5.3 Nível Recursivo: Tabela de Transições da Máquina de Turing

A Máquina de Turing opera com fita infinita e estados $Q = \{q_0, q_{p0}, q_{p1}, q_{m0}, q_{m1}, q_r, q_{re}, q_{vf}, q_{aceita}\}$.

Estado	Lido	Próximo Estado	Escrito	Mov.	Descrição da Ação na Fita
q0	0 / 1	q_hash_0 / q_hash_1	X / Y	R	Marca o bit atual à esquerda e vai procurar o delimitador #
q0	#	q_verifica_fim	#	R	Bloco esquerdo processado. Valida o fim à direita
q_hash_0/1	0 ou 1	q_hash_0/1	Mantenha	R	Avança ignorando os bits não marcados do bloco esquerdo
q_hash_0/1	#	q_match_0 / q_match_1	#	R	Cruza o delimitador central e entra no bloco da direita
q_match_0/1	X ou Y	q_match_0/1	Mantenha	R	Salta células da direita que já foram validadas
q_match_0/1	0 / 1	q_retorna	X / Y	L	Casamento correto do bit! Aplica a marca e inicia retorno
q_retorna	não #	q_retorna	Mantenha	L	Move o cabeçote para a esquerda rebobinando os dados
q_retorna	#	q_retorna_esq	#	L	Cruza o delimitador retornando à área esquerda
q_retorna_esq	X ou Y	q0	Mantenha	R	Localiza a última marcação da esquerda e reinicia o ciclo
q_verifica_fim	— (Vazio)	q_aceita	—	R	Fita perfeitamente casada. Atinge Aceitação Final

6. Implementação do Sistema

6.1 Visão Geral e Eliminação de Condicionais

O ecossistema foi estruturado de forma modular no diretório `src/`, gerando os arquivos `regular.py`, `livre_contexto.py` e `recursiva.py`. Cada um expõe o método unificado `reconhecer(cadeia)`, que retorna a aceitação e a contagem de passos.

Para cumprir a exigência conceitual de simulação pura, a lógica de transição descarta bifurcações condicionais estruturadas em blocos procedimentais de `if/elif/else`. Em vez disso, a computação baseia-se estritamente na consulta a tabelas indexadas compostas por dicionários estáticos. Abaixo, detalha-se o núcleo do algoritmo do PDA:

```
# Excerto demonstrativo extraído de src/livre_contexto.py
class SimuladorPDA:
    def __init__(self):
        self.alfabeto = set("()abcdefghijklmnopqrstuvwxyz0123456789+-* / ")
        self.transicoes = {
            ("q1", "(", "Z"): ("q1", ("X", "Z")),
            ("q1", "(", "X"): ("q1", ("X", "X")),
            ("q1", ")", "X"): ("q1", ()),
        }
        for char in "abcdefghijklmnopqrstuvwxyz0123456789+-* / ":
            self.transicoes[("q1", char, "Z")] = ("q1", ("Z",))
            self.transicoes[("q1", char, "X")] = ("q1", ("X",))

    def receber_transicao(self, est, sym, topo):
        return self.transicoes.get((est, sym, topo), None)

    def reconhecer(self, cadeia):
        pilha = ["Z"]
        passos = 0
        for simbolo in cadeia:
            if simbolo not in self.alfabeto or not pilha:
                return False, passos
            res = self.receber_transicao("q1", simbolo, pilha.pop())
            if res is None:
                return False, passos
            for elem in reversed(res[1]):
                pilha.append(elem)
            passos += 1
        return (len(pilha) == 1 and pilha[0] == "Z"), passos
```

7. Testes e Análise Empírica de Resultados

A execução automatizada da suíte de testes contida no script `src/testes.py` validou as cadeias de caracteres a partir da pasta `teste/`. Foram inspecionados casos exatos, variações de borda e violações estruturais, totalizando 18 baterias completas de verificação formal.

Nível da Linguagem	Cadeia Testada	Esperado	Obtido	Passos
REGULAR (DFA)	123.456.789-00 / 000.000.000-00	True	True	14
REGULAR (DFA)	12.34.56-78 / 123.456.78900	False	False	2 / 11
REGULAR (DFA)	"" (Cadeia Vazia)	False	False	0
LIVRE DE CONTEXTO (PDA)	((x+y)*z) / (((a)))	True	True	9 / 7
LIVRE DE CONTEXTO (PDA)	((a+b) / (a+b))	False	False	5 / 5
LIVRE DE CONTEXTO (PDA)	)	False	False	0
RECURSIVA (MT)	101#101 / 0#0	True	True	32 / 8
RECURSIVA (MT)	#	True	True	1
RECURSIVA (MT)	101#100 / 10#101	False	False	18 / 13

7.1 Comportamento Analítico da Volumetria de Passos

A contagem empírica fornece evidências práticas sobre a complexidade assintótica de cada classe: no modelo Regular (DFA), o avanço ocorre de forma linear. Toda cadeia válida consome 14 passos. No modelo Livre de Contexto (PDA), o consumo mantém proporção exata de **1:1** em relação ao comprimento da string ($O(N)$). No modelo Recursivo (MT), o número de passos revela o salto quadrático $O(N^2)$ decorrente do vaivém pendular do cabeçote (a cadeia válida **101#101** de tamanho 7 demandou 32 passos na fita).

8. Comparação entre os Níveis e Conclusão

O desenvolvimento simultâneo destes simuladores comprova empiricamente o teorema da inclusão estrita da Hierarquia de Chomsky ($LR \subsetneq LLC \subsetneq R$). As disparidades fundamentais residem na capacidade de armazenamento: o DFA retém informação apenas nos estados finitos; o PDA adiciona uma pilha infinita com acesso restrito ao topo (LIFO); a Máquina de Turing elimina restrições introduzindo uma fita de leitura e escrita com acesso aleatório bidirecional.

A lição central de engenharia é a relevância da escolha do modelo mínimo adequado. Embora a Máquina de Turing possua poder computacional absoluto capaz de resolver as três linguagens, utilizá-la para validar um padrão regular como um CPF seria um erro grave de arquitetura, gerando complexidade desnecessária e custo computacional proibitivo. As principais limitações residem na sensibilidade a loops infinitos na MT (mitigada por trava estática em 20.000 passos) e na abordagem destrutiva da fita e da pilha, exigindo cópias em memória em cenários reais.